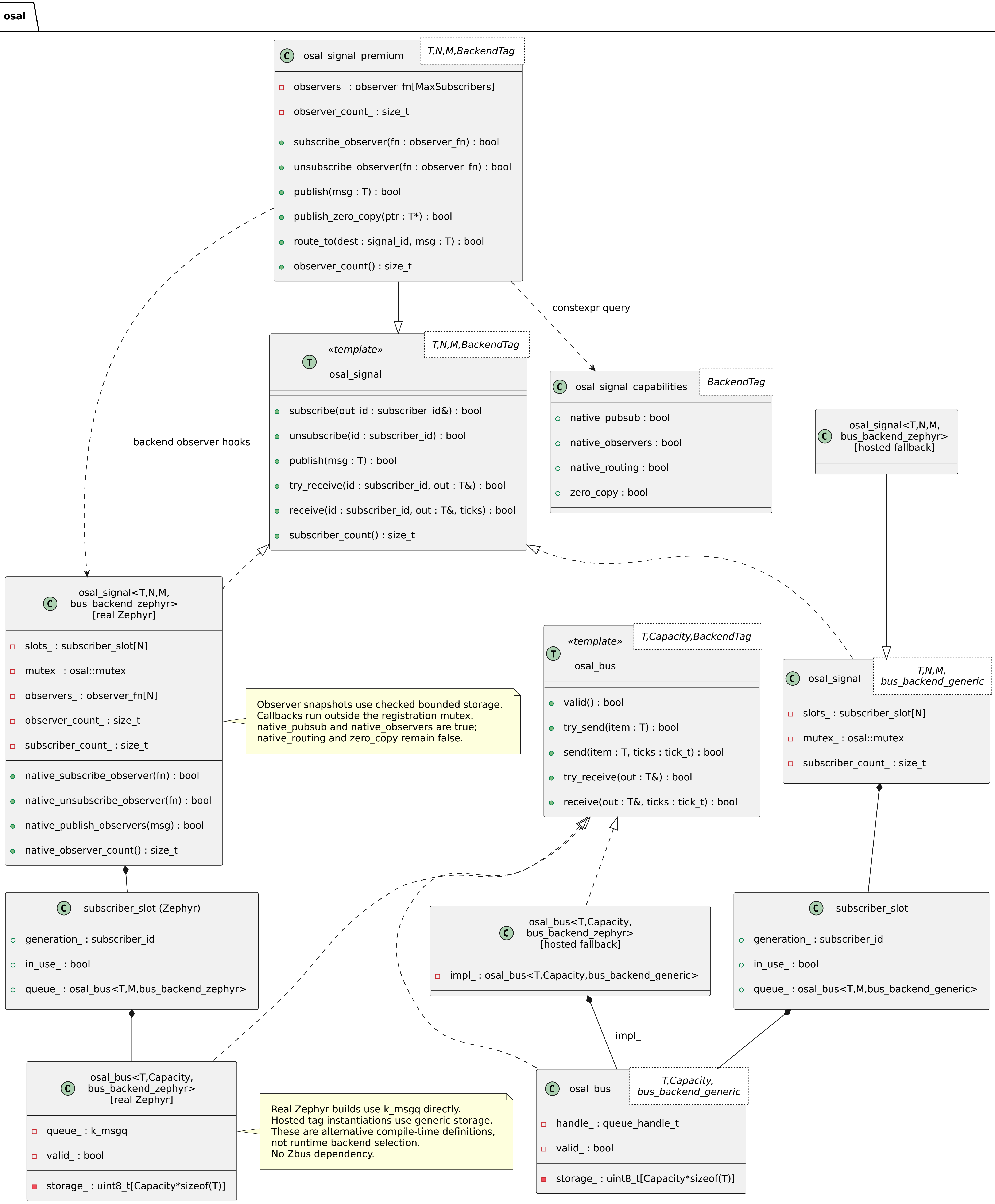


microOSAL Bus + Signal Layer — Class Diagram



«template»

osal_bus

T,Capacity,BackendTag

valid() : bool

try_send(item : T) : bool

send(item : T, ticks : tick_t) : bool

try_receive(out : T&) : bool

receive(out : T&, ticks : tick_t) : bool

osal_signal<T,N,M,
bus_backend_generic>
[hosted fallback]

slots_ : subscriber_slot[N]

mutex_ : osal::mutex

subscriber_count_ : size_t

osal_bus<T,Capacity,
bus_backend_zephyr>
[hosted fallback]

impl_ : osal_bus<T,Capacity,bus_backend_generic>

subscriber_slot (Zephyr)

generation_ : subscriber_id

in_use_ : bool

queue_ : osal_bus<T,M,bus_backend_zephyr>

osal_bus<T,Capacity,
bus_backend_zephyr>
[real Zephyr]

queue_ : k_msgq

valid_ : bool

storage_ : uint8_t[Capacity*sizeof(T)]

osal_bus

T,Capacity,
bus_backend_generic

handle_ : queue_handle_t

valid_ : bool

storage_ : uint8_t[Capacity*sizeof(T)]

subscriber_slot

generation_ : subscriber_id

in_use_ : bool

queue_ : osal_bus<T,M,bus_backend_generic>

Observer snapshots use checked bounded storage. Callbacks run outside the registration mutex. native_pubsub and native_observers are true; native_routing and zero_copy remain false.

Real Zephyr builds use k_msgq directly. Hosted tag instantiations use generic storage. These are alternative compile-time definitions, not runtime backend selection. No Zbus dependency.

backend observer hooks

constexpr query

impl_

osal_signal_premium

«template» osal_signal

osal_signal_capabilities

osal_signal<T,N,M, bus_backend_zephyr> [real Zephyr]

«template» osal_bus

osal_signal<T,N,M, bus_backend_generic> [hosted fallback]

osal_bus<T,Capacity, bus_backend_zephyr> [hosted fallback]

subscriber_slot (Zephyr)

osal_bus<T,Capacity, bus_backend_zephyr> [real Zephyr]

osal_bus

subscriber_slot